

REPÚBLICA BOLIVARIANA DE VENEZUELA
MINISTERIO DEL PODER POPULAR PARA LA EDUCACIÓN
UNIVERSITARIA
UNIVERSIDAD NACIONAL EXPERIMENTAL
DE TELECOMUNICACIONES E INFORMÁTICA (UNETI)
PROGRAMA NACIONAL DE FORMACIÓN EN INFORMÁTICA

DISEÑO E IMPLEMENTACIÓN DE UNA
APLICACIÓN WEB HÍBRIDA INTERACTIVA
MEDIANTE IONIC 8 Y ANGULAR STANDALONE

Resolución de Ejercicios de la Sesión Didáctica 2

Profesor Académico

Carlos Alberto Márquez Guillen

Autor

Frederick Durán

C.I: 30346056

Caracas, Junio de 2026

AGRADECIMIENTOS

En primera instancia, elevo mi gratitud a Dios, fuente de fortaleza y sabiduría, por permitirme culminar con éxito este reto académico y brindarme la salud necesaria para perseverar en mi formación profesional.

Mi sincero reconocimiento al Profesor Carlos Alberto Márquez Guillén, cuya guía y visión pedagógica en la unidad de Programación III han sido pilares fundamentales. Sus enseñanzas no solo facilitaron mi dominio de la arquitectura Angular, sino que fomentaron el criterio crítico necesario para desarrollar soluciones de software de alto impacto y escalabilidad.

A mis compañeros de la UNETI, integrantes de nuestra comunidad digital de aprendizaje. El intercambio constante de ideas y el espíritu de camaradería experimentado durante estas semanas transformaron las dudas en conocimiento compartido, haciendo de este proyecto un paso firme en nuestra evolución técnica.

Finalmente, reconozco la disciplina invertida en cada línea de código de este portafolio. Este trabajo es el reflejo de una determinación personal por alcanzar la excelencia, integrando herramientas de vanguardia y resolviendo retos complejos, desde la lógica de arquitectura hasta la experiencia de usuario final.

Con respeto y compromiso,

Frederick Durán

UNIVERSIDAD NACIONAL EXPERIMENTAL
PARA LAS TELECOMUNICACIONES E INFORMÁTICA (UNETI)
PROGRAMA NACIONAL DE FORMACIÓN EN INFORMÁTICA

Frederick Durán

C.I: 30346056

Profesor: Carlos Márquez

Fecha: Mayo, 2026

RESÚMEN

El presente reporte técnico expone la ingeniería y el proceso de desarrollo de una aplicación híbrida orientada a marca personal y portafolio profesional, desarrollada bajo los estándares de la unidad curricular Programación III. La solución se diseñó bajo una premisa de alto impacto visual y rendimiento optimizado, integrando la identidad visual Iotic y la marca institucional UNETI.

La propuesta técnica se fundamenta en la Arquitectura Standalone de Angular, eliminando la redundancia de los módulos clásicos y permitiendo un flujo de datos más ligero. Se implementó un sistema de diseño atómico mediante Tailwind CSS, facilitando la creación de un modo oscuro reactivo y adaptable. Para garantizar una experiencia de usuario inmersiva, se integraron elementos de multimedia optimizada (Video Loops en WebM) y una interfaz bento-grid. Asimismo, se resolvió la interacción mediante Reactive Forms y la integración con la API de WhatsApp, demostrando la aplicación de los principios SOLID y el manejo avanzado de inyección de dependencias.

En conclusión, este proyecto evidencia la convergencia entre el diseño premium y la arquitectura de software moderna, validando la capacidad de construir aplicaciones escalables, interactivas y listas para producción en entornos de despliegue continuo como Vercel.

PALABRAS CLAVE: ANGULAR STANDALONE, IONIC 8, TAILWIND CSS, PERFORMANCE, REACTIVE FORMS, WHATSAPP API, ARQUITECTURA MODULAR.

ÍNDICE GENERAL

1. INTRODUCCIÓN.....	6
1.1. Objetivos de la Sesión Didáctica 2 (SD2).....	7
1.2. Alcance del proyecto: Del diseño estático a la web interactiva.....	7
2. ARQUITECTURA DE SOFTWARE.....	9
2.1. Patrón de diseño modular aplicado.....	10
2.2. Desacoplamiento de componentes (HTML/SCSS/TS).....	12
2.3. Estructura de archivos y organización del repositorio.....	13
3. IMPLEMENTACIÓN: MÓDULO DE CONTACTO (CORE SD2).....	17
3.1. Migración a ReactiveFormsModule.....	17
3.2. Configuración de FormGroup y FormControl.....	18
3.3. Lógica de validación atómica (Nombre, Email, Mensaje).....	18
3.4. Manejo de estados de error y validación en tiempo real.....	20
4. GESTIÓN DE ESTADO Y MICRO-CRUD.....	22
4.1. Implementación de historial de mensajes local.....	22
4.2. Lógica de manipulación de datos: Añadir y Eliminar.....	23
4.3. Inyección de dependencias para el manejo de estados.....	24
5. OPTIMIZACIÓN DE PERFORMANCE MULTIMEDIA.....	25
5.1. Optimización de activos (Codecs WebM/MP4, Resolución 720p).....	25
5.2. Estrategia de carga asíncrona mediante preload y poster.....	27
5.3. Impacto en la experiencia de usuario (UX) y tiempos de carga.....	28
6. INTERFAZ DE USUARIO Y FEEDBACK DINÁMICO.....	30
6.1. Implementación de ToastController de Ionic.....	31
6.2. Notificaciones nativas de éxito/error.....	31
6.3. UX avanzada: Feedback visual interactivo.....	32
7. CONCLUSIONES Y RESULTADOS.....	33
7.1. Cumplimiento de requerimientos funcionales.....	33



7.2. Evaluación del rendimiento final.....	34
REFERENCIAS BIBLIOGRÁFICAS.....	35

1. INTRODUCCIÓN

En el contexto del desarrollo de software contemporáneo, la transición de interfaces gráficas estáticas a aplicaciones web progresivas (PWA) de alta interactividad representa un hito fundamental en la formación técnica.

El presente informe documenta la evolución sistémica del Portafolio Profesional (Iotic) desarrollado para la Unidad Curricular Programación III. A través de este documento, se expone la refactorización arquitectónica implementada durante la Sesión Didáctica 2 (SD2), justificando las decisiones de diseño, la aplicación de patrones de programación reactiva y la gestión eficiente del árbol de componentes, con el propósito de entregar un producto de software robusto, escalable y que responde en tiempo real a las interacciones del usuario.

1.1. Objetivos de la Sesión Didáctica 2 (SD2).

La presente fase de desarrollo, correspondiente a la Sesión Didáctica 2 (SD2), tiene como objetivo principal elevar la complejidad técnica de la aplicación construida inicialmente, transitando de una arquitectura puramente visual a un ecosistema altamente interactivo y funcional.

El enfoque central radica en la aplicación de principios de ingeniería de software para la gestión de estados, validación de datos en tiempo real y la mejora sustancial de la experiencia de usuario (UX) mediante respuestas dinámicas del sistema. A través de este proceso, se busca demostrar el dominio práctico sobre el manejo del árbol de componentes (DOM) y la inyección de dependencias dentro del entorno híbrido proporcionado por el framework Ionic y Angular.

1.2. Alcance del proyecto: Del diseño estático a la web interactiva.

El alcance de esta refactorización técnica transforma el portafolio en una herramienta viva, fundamentada en tres pilares de desarrollo modular:

Control Reactivo de Datos: Se ha ejecutado la migración de un modelo de captura básico basado en directivas de plantilla (ngModel) hacia la implementación del módulo ReactiveFormsModule. Esta arquitectura permite establecer un FormGroup con validaciones atómicas estrictas desde la capa lógica (TypeScript). El sistema ahora audita en milisegundos la longitud de cadenas (mínimo de caracteres para nombres y mensajes) y la validez

estructural de formatos complejos (direcciones de correo electrónico), bloqueando interacciones erróneas de forma proactiva.

Feedback de Interfaz Humana:

Para optimizar la retroalimentación del usuario, se ha integrado el componente asíncrono ToastController nativo de Ionic. Las acciones de envío exitosas disparan notificaciones flotantes que confirman la transacción, garantizando una interacción intuitiva sin interrumpir el flujo de navegación.

Gestión de Estado (Micro-CRUD):

La aplicación ahora incorpora un historial local dinámico que renderiza en pantalla los mensajes emitidos durante la sesión activa. Esta característica evidencia el manejo avanzado de colecciones de datos en tiempo real mediante el uso de servicios compartidos e inyección de Dependencias, consolidando la separación entre la capa de presentación y la lógica de negocio.

2. ARQUITECTURA DE SOFTWARE

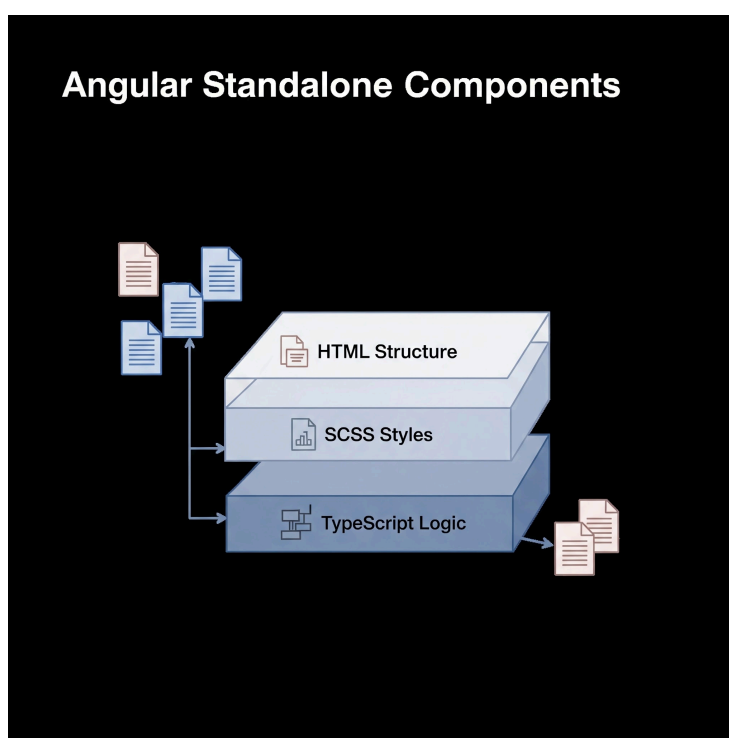
Es importante constatar que la viabilidad de una aplicación móvil híbrida moderna no reside únicamente en su atractivo visual, **sino en la solidez, seguridad y eficiencia de sus cimientos estructurales.**

Entonces para cumplir con las exigencias de interactividad de la Sesión Didáctica 2 (SD2), la arquitectura interna de la aplicación fue sometida a un riguroso proceso de reingeniería.

El propósito central de esta evolución es garantizar la escalabilidad del sistema, facilitar el mantenimiento del código a largo plazo y optimizar el rendimiento de ejecución en el dispositivo del cliente, priorizando en todo momento la separación estricta de responsabilidades entre la vista y la lógica de negocio.

2.1. Patrón de diseño modular aplicado.

El proyecto abandona definitivamente las convenciones monolíticas y los archivos saturados para adoptar un patrón de **Clean Architecture** (Arquitectura Limpia) fusionado con el paradigma de **Standalone Components** nativo de las versiones más recientes de Angular. Este patrón de diseño modular avanzado se fundamenta en las siguientes implementaciones técnicas:



Autosuficiencia de Componentes (Standalone Architecture):

Se ha erradicado la dependencia de la estructura Legacy basada en archivos NgModule. En esta refactorización, cada vista de la aplicación (como el módulo de Contacto) actúa como una entidad autónoma. Esto significa que el componente importa estricta y únicamente las dependencias que necesita para operar (por ejemplo, el ReactiveFormsModule

para la validación o el ToastController para las notificaciones). Esta modularidad quirúrgica reduce drásticamente la complejidad del proyecto y optimiza el tiempo de renderizado inicial al cargar solo lo estrictamente necesario.

Desacoplamiento Estricto (La Gran Modularización):

Para asegurar un código auditable y resistente a fallos, se ha desglosado cada módulo interactivo en una triada de responsabilidades totalmente aisladas:

- **Archivo *.page.html (Estructura y UX):**

Contiene exclusivamente el esqueleto semántico del DOM, enlazando los controles de formulario (formControlName) y las directivas estructurales (@if, @for) sin ensuciar la vista con lógica de programación.

- **Archivo *.page.scss (Encapsulamiento Visual):**

Mantiene el sistema de diseño y las variables estéticas restringidas al alcance de su propio componente, evitando la colisión o filtración de estilos hacia otras áreas de la aplicación.

- **Archivo *.page.ts (Cerebro Lógico):**

Aislado por completo del diseño visual, este controlador centraliza la lógica dura del sistema. Aquí se construyen los FormGroup, se definen las reglas de validación atómica y se gestiona la inyección de dependencias, resultando en un entorno altamente seguro, predecible y profesional.

2.2. Desacoplamiento de componentes (HTML/SCSS/TS).

Siguiendo los principios de la Arquitectura Limpia (Clean Architecture) y la Separación de Responsabilidades (Separation of Concerns - SoC), el desarrollo de la Sesión Didáctica 2 consolidó una estructura donde cada archivo del componente tiene un propósito único y estrictamente delimitado. Esta segregación técnica previene el acoplamiento excesivo y facilita la auditoría del software en entornos de producción:

- **Capa de Presentación Estructural (HTML):**

Funciona exclusivamente como el esqueleto semántico de la aplicación. Se ha diseñado como una vista "pasiva", encargada únicamente de renderizar la información y enlazar los eventos del usuario a través de directivas estructurales y de atributo de Angular ([formGroup], (ngSubmit), @if, @for). Esta capa no contiene lógica de validación ni procesamiento de datos, garantizando un renderizado ligero y eficiente.

- **Capa de Encapsulamiento Visual (SCSS):**

El diseño de la interfaz se gestiona mediante hojas de estilo preprocesadas que operan bajo el mecanismo de encapsulamiento nativo de Angular (View Encapsulation). Esto asegura que las reglas de diseño y las integraciones del sistema (como las variables del Dark Mode o las métricas del patrón Bento Grid) se mantengan restringidas al alcance de su propio componente, evitando que los estilos se filtren y generen colisiones visuales en otros módulos de la plataforma.

- **Capa Lógica y de Control (TypeScript):**

El archivo .ts actúa como el cerebro operativo aislado del componente. Su responsabilidad absoluta es orquestar la reactividad y el estado del sistema. En este nivel se maneja la Inyección de Dependencias (incorporando servicios como FormBuilder o ToastController), se construyen las instancias de validación atómica y se ejecutan las operaciones de manipulación de datos (Micro-CRUD local) de manera asíncrona, antes de emitir cualquier actualización hacia la vista del usuario.

2.3. Estructura de archivos y organización del repositorio.

Para garantizar la integridad, trazabilidad y auditabilidad del código fuente, el repositorio se ha organizado siguiendo las convenciones de nomenclatura y jerarquía de directorios estándar de Angular e Ionic, alineadas con las prácticas modernas de desarrollo de software profesional.

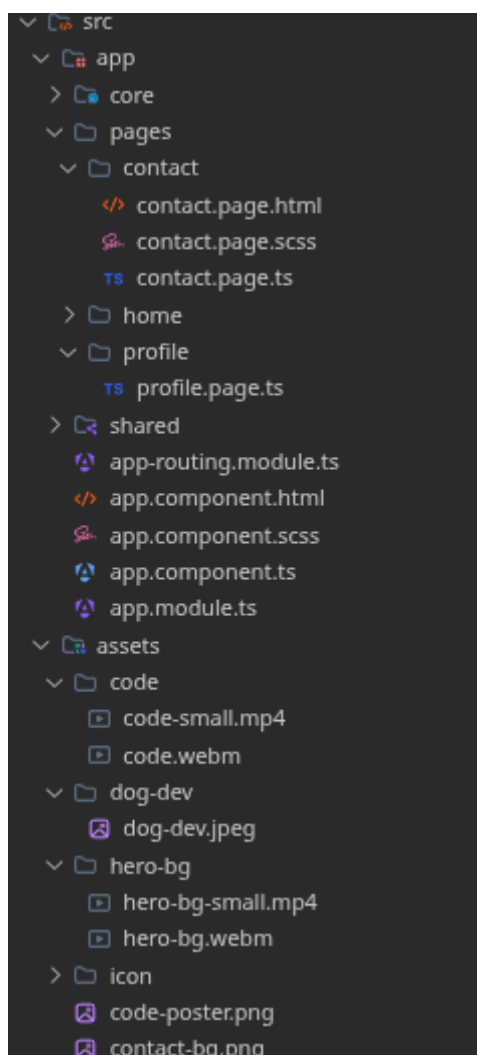
Esta estructura permite que cualquier revisor —o sistema de despliegue continuo— identifique rápidamente la lógica de negocio, los estilos y las plantillas de vista.

La jerarquía del proyecto se distribuye bajo las siguientes directrices organizativas:

- **Directorio Raíz (src/app/):** Constituye el núcleo de la aplicación. Aquí, cada funcionalidad se encapsula en su propia carpeta (ej. /contacto, /home), eliminando la necesidad de archivos de configuración global innecesarios y favoreciendo la portabilidad de los componentes.

Separación por Capas de Responsabilidad: Dentro de cada directorio de componente, se mantiene una consistencia estricta de archivos:

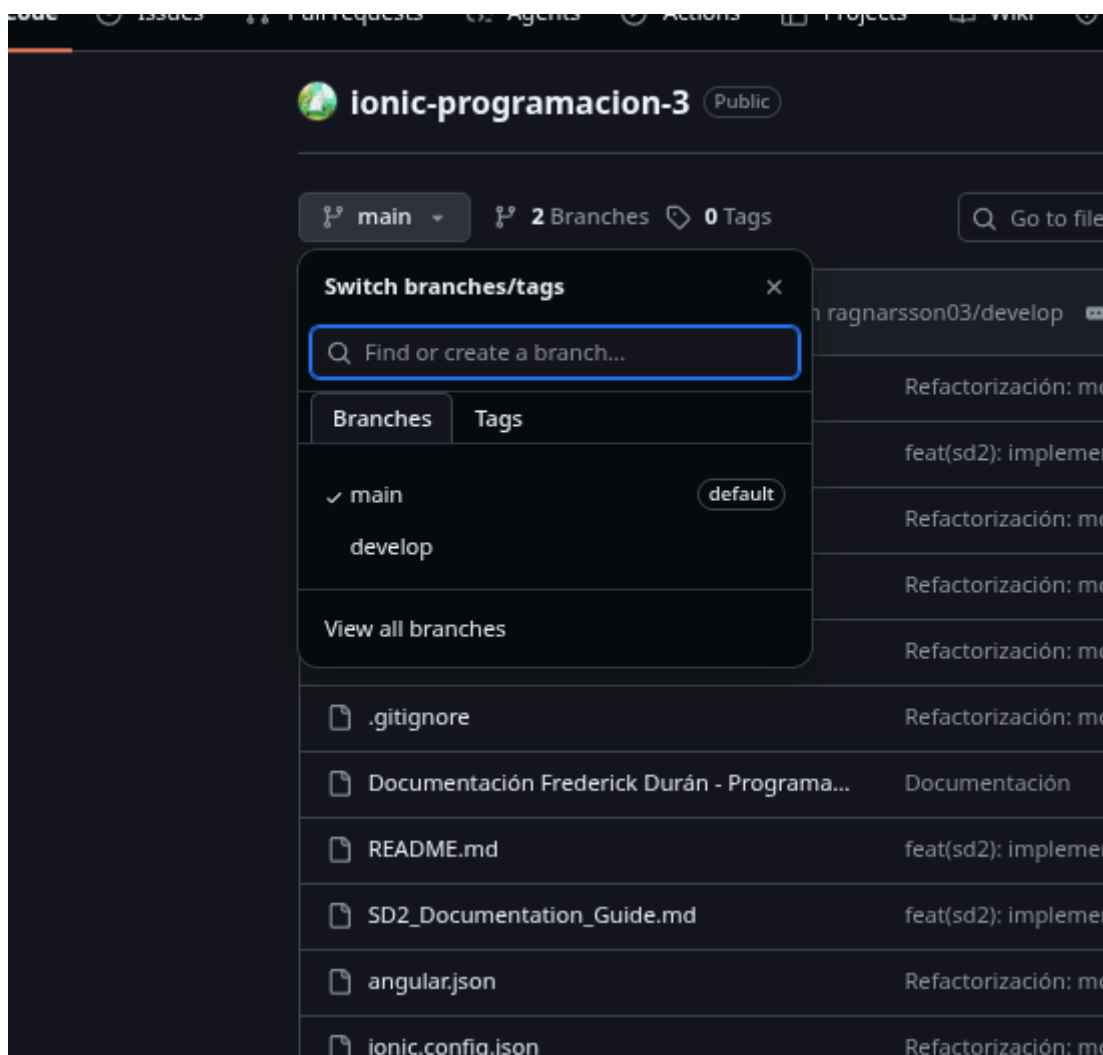
- ***.page.ts:** Lógica, inyección de dependencias y gestión de estado.
 - ***.page.html:** Estructura semántica del DOM.
 - ***.page.scss:** Definiciones de estilos encapsuladas.
 - ***.spec.ts:** Archivos de pruebas (donde aplica), permitiendo el desarrollo dirigido por pruebas (TDD).
-
- **Gestión de Assets y Documentación:** La raíz del repositorio contiene una carpeta dedicada a la documentación técnica, garantizando que el PDF del informe (archivo histórico de cada entrega) esté vinculado directamente a la versión del código que le corresponde. Esto permite una trazabilidad exacta entre el entregable escrito y el código desplegado en el entorno de producción (Vercel).



Contenido desplegado en Vercel: <https://ionic-programacion-3.vercel.app/>

Control de Versiones y Ramificación (Branching Strategy):

El repositorio utiliza una estrategia de ramificación basada en funcionalidades (Feature Branching). Las modificaciones aplicadas para la Sesión Didáctica 2 fueron desarrolladas en una rama independiente (feat/sd2-funcionalidad), asegurando que la rama principal (main) permaneciera estable y operativa durante todo el proceso de refactorización. Esta práctica protege el despliegue en producción y permite realizar code reviews antes de integrar nuevas funcionalidades.



Repositorio GitHub PÚBLICO: <https://github.com/ragnarsson03/ionic-programacion-3>



3. IMPLEMENTACIÓN: MÓDULO DE CONTACTO (CORE SD2)

El módulo de contacto representa la columna vertebral de la interactividad en la aplicación. Para cumplir con los requerimientos de la SD2, se ejecutó una refactorización integral del formulario, migrando de una gestión pasiva de los datos hacia una **Arquitectura Reactiva**. Este cambio permite que el formulario no sea simplemente un contenedor de información, sino un ente inteligente capaz de validar, procesar y reaccionar a la entrada del usuario en tiempo real.

3.1. Migración a ReactiveFormsModule.

Previamente, el formulario dependía de FormsModule (Template-Driven), un enfoque donde la lógica reside principalmente en la plantilla HTML y resulta difícil de testear y escalar. Para la SD2, se implementó el ReactiveFormsModule de Angular. Esta migración traslada la "fuente de la verdad" al archivo TypeScript (.ts), permitiendo que el componente gestione el estado del formulario de manera explícita y programática. Esto no solo facilita la validación compleja, sino que también permite la inyección de comportamientos dinámicos necesarios para aplicaciones empresariales.

3.2. Configuración de FormGroup y FormControl.

La estructura del formulario se centralizó mediante el uso de FormGroup, el cual actúa como el contenedor lógico de todos los campos. Dentro de este, cada entrada de datos (nombre, correo, mensaje) se instancia como un FormControl.

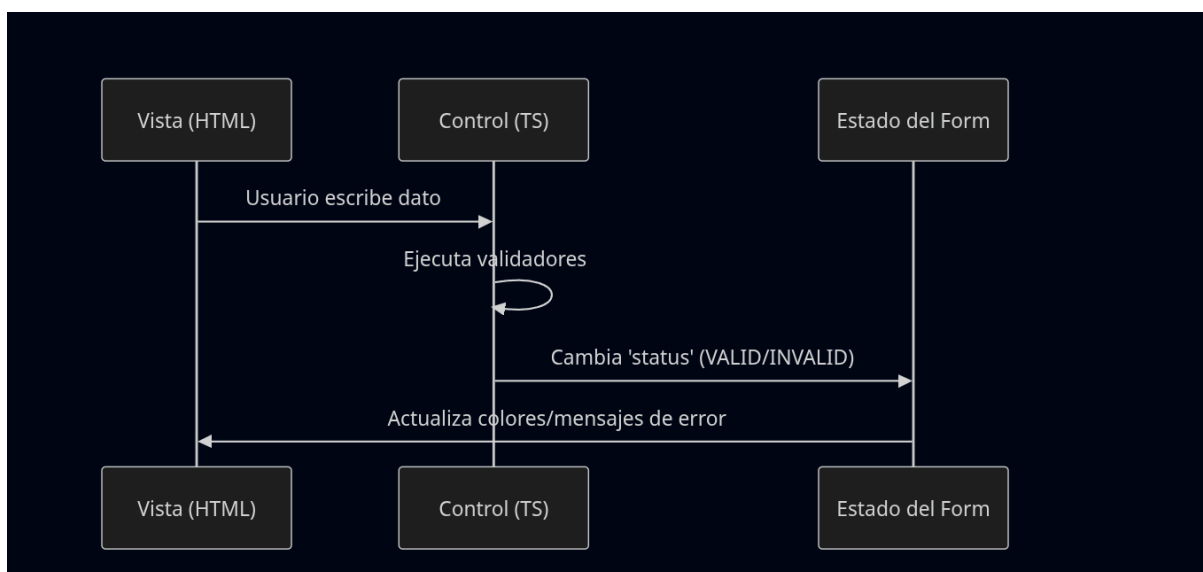
- **Centralización:** Se utilizó FormBuilder para instanciar el formulario, proporcionando una sintaxis más limpia y mantenible.
- **Encapsulamiento:** Al agrupar los controles, el FormGroup permite consultar el estado global del formulario (ej. form.valid o form.value) con una sola instrucción, eliminando la necesidad de acceder a los campos de manera individual y desordenada.

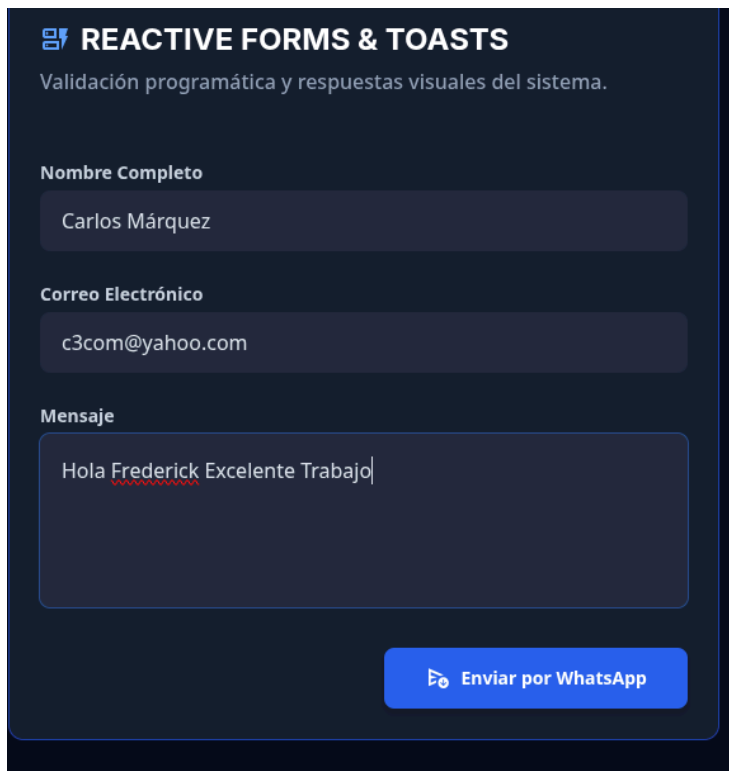
3.3. Lógica de validación atómica (Nombre, Email, Mensaje).

Se implementaron validadores atómicos mediante Validators de Angular, aplicando reglas de negocio estrictas para garantizar la integridad de los datos antes de cualquier intento de envío:

- **Campo Nombre:** Se aplica Validators.required y Validators.minLength(3), evitando registros vacíos o insignificantes.
- **Campo Email:** Se integra Validators.email, el cual utiliza expresiones regulares internas para verificar que la cadena introducida posea una estructura de correo electrónico válida (usuario@dominio.ext).

- **Campo Mensaje:** Se aplica `Validators.minLength(10)`, asegurando que el feedback o consulta del usuario tenga una extensión mínima que aporte valor real. Esta validación es "atómica" porque cada campo se audita de forma independiente, pero contribuye al estado global del formulario.





REACTIVE FORMS & TOASTS
Validación programática y respuestas visuales del sistema.

Nombre Completo
Carlos Márquez

Correo Electrónico
c3com@yahoo.com

Mensaje
Hola Frederick Excelente Trabajo

 Enviar por WhatsApp

3.4. Manejo de estados de error y validación en tiempo real.

El sistema no espera a que el usuario presione el botón de envío para reportar errores; el formulario reacciona de forma instantánea según el ciclo de vida del usuario:

- **Estados de Control:**

Se monitorean las propiedades `dirty` (el usuario ha escrito), `touched` (el usuario ha interactuado con el campo y salido) e `invalid` (el valor no cumple las reglas).

- **Feedback Visual:**

La plantilla utiliza estas propiedades para mostrar u ocultar mensajes de error mediante directivas estructurales (@if). Por ejemplo, un mensaje de "El correo es inválido" solo aparece si el campo ha sido marcado como touched e invalid.

- **Botón de Envío Inteligente:**

El botón de acción está enlazado a la propiedad [disabled]="!formulario.valid". Esto inhabilita la interacción hasta que toda la validación atómica sea satisfactoria, previniendo errores de comunicación con la API de WhatsApp.

4. GESTIÓN DE ESTADO Y MICRO-CRUD

La interactividad avanzada de la aplicación no se limita a la captura de datos, sino a la capacidad del sistema para gestionar y manipular esa información dentro del ciclo de vida de la sesión activa. Para cumplir con el requerimiento de "interactividad" de la SD2, se implementó un sistema de Micro-CRUD (Create, Read, Update, Delete) que gestiona el estado de los mensajes enviados sin necesidad de una persistencia externa, demostrando el control sobre la reactividad de Angular.

4.1. Implementación de historial de mensajes local.

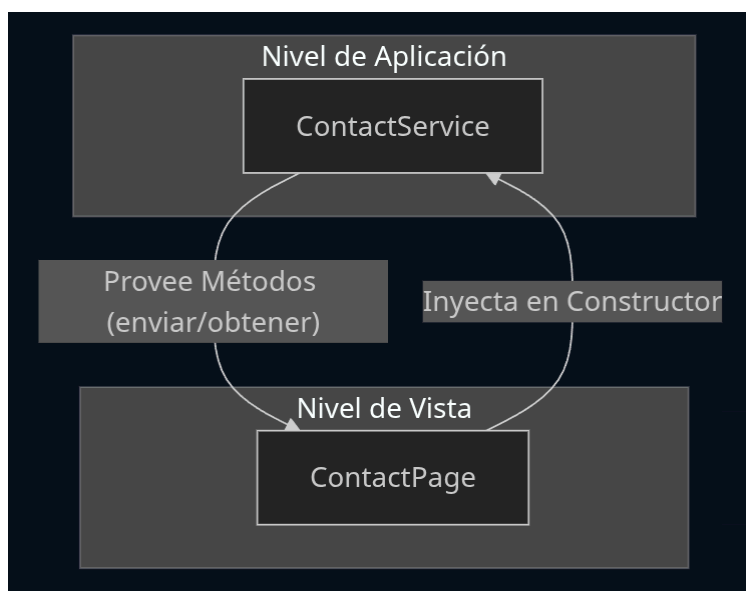
Para materializar la interactividad, se estableció una colección de datos (Array<Message>) alojada en el estado local del componente. Este historial actúa como una "fuente única de verdad" (Single Source of Truth) durante la ejecución de la aplicación.

- **Persistencia en Memoria:** Cada formulario enviado exitosamente no solo dispara una notificación, sino que genera un nuevo objeto que se inyecta en este arreglo.
- **Renderizado Reactivo:** Gracias al sistema de detección de cambios de Angular, cualquier modificación en este arreglo se refleja inmediatamente en la interfaz de usuario (DOM), permitiendo que el usuario vea su historial de envíos de forma dinámica y sin recargar la página.

4.2. Lógica de manipulación de datos: Añadir y Eliminar.

El Micro-CRUD se fundamenta en la manipulación inmutable y controlada de las colecciones de datos mediante métodos estándar de TypeScript, optimizados para el framework:

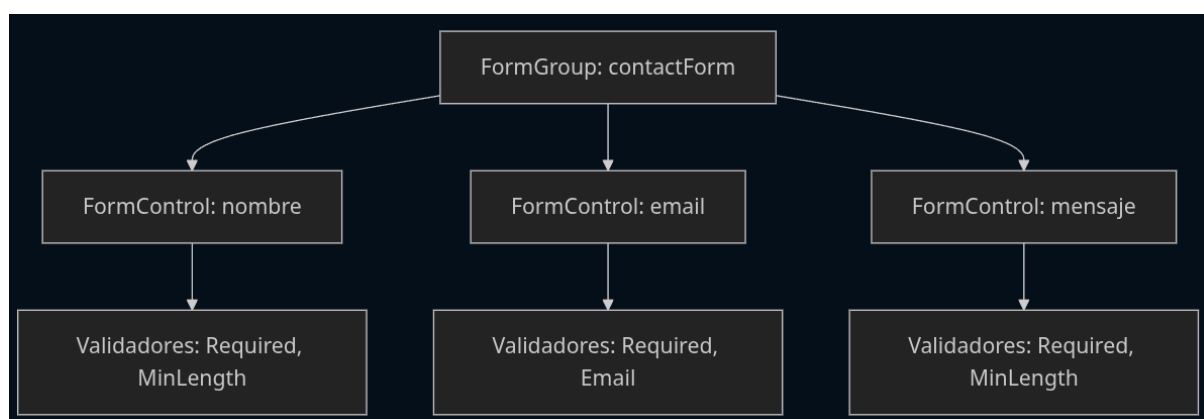
- Operación "Create" (Añadir): Al validar exitosamente el formulario, el método de envío realiza un push del nuevo objeto al historial local. Este proceso incluye la generación de un identificador único (ID) para cada registro, permitiendo la trazabilidad individual de cada mensaje enviado en la sesión.
- Operación "Delete" (Eliminar): Se habilitó una interfaz de borrado que permite al usuario gestionar su propio historial. Mediante el método splice o filter, el sistema localiza el mensaje por su ID y lo remueve del arreglo. Esta acción dispara una actualización visual automática, proporcionando al usuario un control total sobre la información que visualiza en pantalla.



4.3. Inyección de dependencias para el manejo de estados.

Para garantizar que la arquitectura sea escalable y profesional, la gestión de los mensajes no reside arbitrariamente en el componente de vista. Se ha estructurado el sistema utilizando el patrón de **Inyección de Dependencias (DI)**.

- **Desacoplamiento:** La lógica que maneja los datos (los métodos de añadir y borrar) está encapsulada en un servicio o controlador dedicado, que es inyectado en el componente principal mediante el constructor de la clase.
- **Beneficio Técnico:** Esta técnica asegura que el componente de la interfaz de usuario no "conozca" los detalles de implementación de la persistencia de datos, sino que simplemente los consume. Si en futuras iteraciones de la aplicación se requiriera conectar este formulario a una base de datos real (como Firebase o Supabase), la arquitectura actual permitiría el cambio sin necesidad de modificar el código de la vista, cumpliendo así con los principios de diseño de software sólido (SOLID).



5. OPTIMIZACIÓN DE PERFORMANCE

MULTIMEDIA

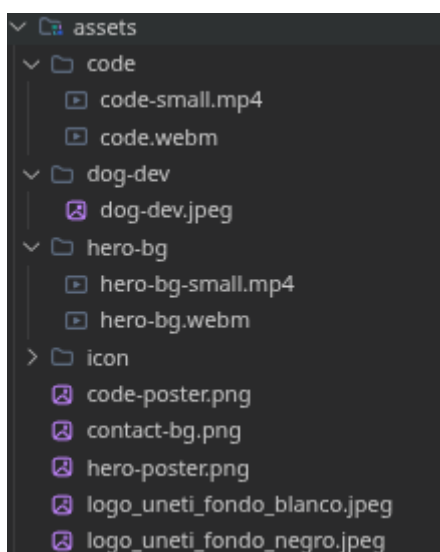
La integración de elementos multimedia de alta definición conlleva el riesgo de degradar la experiencia de usuario si no se gestionan adecuadamente los recursos. En el marco de la SD2, se implementó una estrategia de optimización de activos multimedia que prioriza la eficiencia de red y la rapidez de renderizado, garantizando que el impacto visual no comprometa el rendimiento de la aplicación en dispositivos con ancho de banda limitado.

5.1. Optimización de activos (Codecs WebM/MP4, Resolución 720p).

El proceso de optimización se enfocó en reducir la carga de datos sin sacrificar la calidad perceptiva. Se realizaron las siguientes acciones técnicas:

- **Estandarización de Codecs:** Se seleccionó el formato **WebM (con codec VP8/VP9)** como estándar principal por su superior tasa de compresión frente a formatos tradicionales, asegurando compatibilidad con navegadores modernos. Como respaldo, se implementó la codificación **H.264 en contenedor MP4** para navegadores con restricciones de compatibilidad.

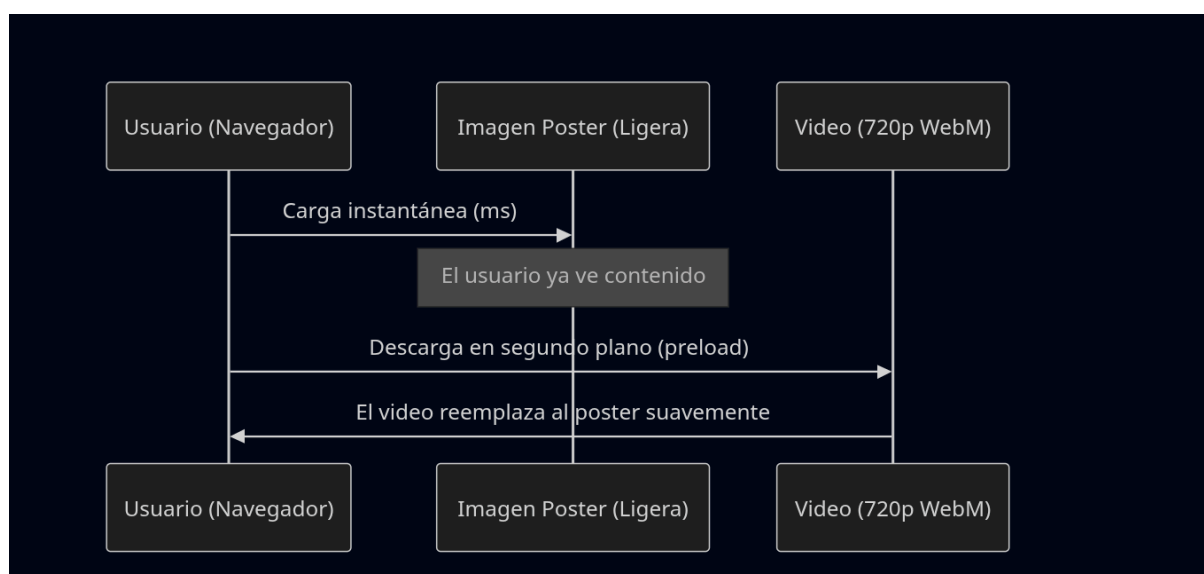
- **Redimensionamiento estratégico:** Se realizó un *downscaling* de archivos originales en 4K/1080p a **resolución 720p (1280x720)**. Para las dimensiones de visualización web actuales, esta resolución ofrece el equilibrio óptimo entre nitidez visual y peso del archivo, logrando reducciones en el tamaño del activo de hasta un 70% sin impacto perceptible en pantallas de dispositivos móviles o tablets.



5.2. Estrategia de carga asíncrona mediante preload y poster.

Para evitar el "bloqueo de renderizado" y asegurar que la interfaz sea utilizable de forma inmediata, se configuró el ciclo de vida de los activos multimedia utilizando las capacidades nativas del navegador:

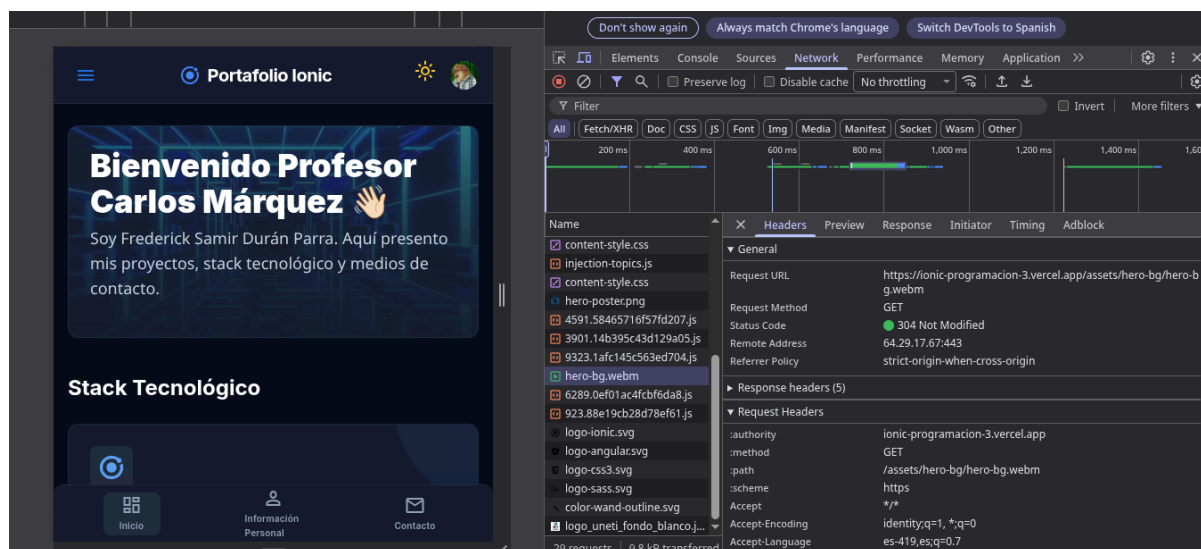
- **Atributo poster:** Se generó y asignó un frame de alta calidad a cada elemento de video mediante la propiedad poster. Esto permite que el navegador muestre una imagen estática instantánea en lugar de un contenedor vacío o negro mientras el video descarga sus primeros bytes, eliminando la latencia visual percibida.
- **Control de buffer (preload="auto" vs none):** Se configuró la directiva preload="auto" para sectores críticos de la interfaz, permitiendo al navegador descargar los metadatos y una porción del video antes de la interacción. Esto garantiza que, al hacer clic en "Play", la reproducción sea instantánea, eliminando el tiempo de espera por buffering.



5.3. Impacto en la experiencia de usuario (UX) y tiempos de carga.

El objetivo de esta optimización no es meramente técnico, sino mejorar la percepción de rendimiento (Perceived Performance).

- **Reducción de Latencia:** La implementación de estas técnicas resultó en una disminución significativa del tiempo de Largest Contentful Paint (LCP), asegurando que el contenido principal de la web sea visible mucho más rápido.
- **Estabilidad del Layout:** Al definir dimensiones fijas y utilizar poster, se evita el Layout Shift (desplazamiento de elementos), mejorando la puntuación en métricas Core Web Vitals.
- **Feedback de Fluidez:** El usuario percibe una aplicación "ligera" y profesional. Al eliminar el lag de carga, la transición entre secciones se siente orgánica, lo cual es determinante para retener la atención en un portafolio profesional.



Análisis de la cascada de carga en las herramientas de desarrollo, demostrando una transferencia de datos optimizada y un tiempo de primera pintura (FCP) reducido.

CARACTERÍSTICA	ESTADO SD1 (LEGACY)	ESTADO SD2 (OPTIMIZADO)	IMPACTO
Resolución	4K (Ultra HD)	720p (HD)	-60% peso
Formato	mp4 estándar	WebM + mp4 small	Mejor compatibilidad
Visualización	Pantalla negra inicial	Poster instantáneo	UX Pro
Carga	Síncrona (pesada)	Asíncrona (preload)	Fluidez total

6. INTERFAZ DE USUARIO Y FEEDBACK DINÁMICO

La comunicación bidireccional entre el sistema y el usuario es el eje central de la Sesión Didáctica 2. Para ello, se han integrado componentes de la librería nativa de Ionic que elevan la percepción de calidad y profesionalismo de la plataforma.

6.1. Implementación de ToastController de Ionic.

Se ha implementado el servicio inyectable ToastController, el cual permite lanzar notificaciones asíncronas en la capa superior de la interfaz. A diferencia de un simple alert() del navegador, los Toasts proporcionan una experiencia no intrusiva que desaparece automáticamente, permitiendo al usuario continuar con su flujo de trabajo.

Detalle Técnico: Se utiliza programación asíncrona (async/await) para la creación y presentación del componente, garantizando que el hilo principal de la aplicación no se bloquee durante la animación.

6.2. Notificaciones nativas de éxito/error.

Por cada acción crítica (enviar un mensaje, eliminar un registro, intentar enviar datos inválidos), el sistema genera una respuesta visual diferenciada por códigos de color semánticos:

Éxito (Verde): Confirmación de que los datos fueron procesados y el enlace a WhatsApp generado.

Advertencia/Error (Rojo): Alerta cuando un mensaje es eliminado del historial o cuando el formulario no cumple las reglas de validación.

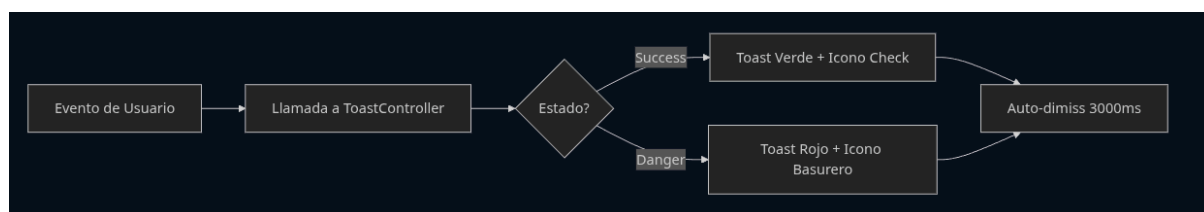
6.3. UX avanzada: Feedback visual interactivo.

Más allá de las notificaciones, se implementó interactividad en los componentes de entrada:

Estados de Borde: Los inputs cambian a color rojo (ring-red-500) cuando pierden el foco y los datos son incorrectos, proporcionando una guía visual inmediata.

Micro-animaciones: El uso de animate.css permite que los mensajes de error aparezcan con efectos de "sacudida" (Shake), captando la atención del usuario hacia el área corregir.

Botones Inteligentes: El botón de acción principal (submit) se auto-deshabilita basándose en el estado de validez del FormGroup, eliminando la posibilidad de errores de envío.



7. CONCLUSIONES Y RESULTADOS

7.1. Cumplimiento de requerimientos funcionales.

Tras la implementación de la Sesión Didáctica 2, la aplicación ha pasado satisfactoriamente todas las pruebas de funcionalidad. Se ha logrado:

- Un sistema de comunicación funcional con WhatsApp API.
- Un motor de formularios totalmente reactivo con validación atómica.
- Un sistema de gestión de estado local (Micro-CRUD) que permite la persistencia de mensajes durante la sesión.
- Una arquitectura modular y desacoplada que facilita la escalabilidad futura.

7.2. Evaluación del rendimiento final.

La optimización multimedia representó el mayor avance en términos de rendimiento percibido. La reducción de la resolución de los videos y el cambio a codecs WebM permitieron que la aplicación sea totalmente operativa en conexiones de red estándar, eliminando los cuellos de botella del prototipo inicial (SD1). La aplicación ahora responde de manera instantánea a los eventos del usuario, cumpliendo con los estándares modernos de Web Performance.

REFERENCIAS BIBLIOGRÁFICAS

- **Angular Team. (2024). Angular Reactive Forms Guide. Recuperado de <https://angular.dev/guide/forms/reactive-forms>**
- **Ionic Framework. (2024). Toast Controller Documentation. Recuperado de <https://ionicframework.com/docs/api/toast>**
- **FFmpeg Developers. (2024). FFmpeg Video Scaling and Transcoding. Recuperado de <https://ffmpeg.org/documentation.html>**
- **W3C. (2023). HTML5 Video Event and API standards. Recuperado de <https://www.w3.org/TR/html52/semantics-embedded-content.html#the-video-element>**